Lahore University of Management Sciences

Syed Babar Ali School of Science and Engineering

AI651 — Deep Learning for Space, Time and Graphs

Assignment 2 Report

Student Name: Faaiz Khan

Student ID: 25280051

GitHub Repository Link:

`https://github.com/FaaizKhan1999/dl4stg-assignments`

Question 1 — River Forecasting Proposal

Each station attends to its upstream sources at learned, state-dependent lags, adds announced releases, and outputs level quantiles at S1–S4 for $t_0{+}10, \ldots, t_0{+}60$ min.

1. Architecture diagram

B = issuance times, $H{=}72$ history steps, $K{=}6$ horizon steps, $N{=}7$ nodes $\{Q, \text{S1..S4}, \text{T1}, \text{TU}\}$ (Q = reservoir, TU = virtual node for the unmonitored tributary), $d{=}64$, $\Delta{=}10$ min.

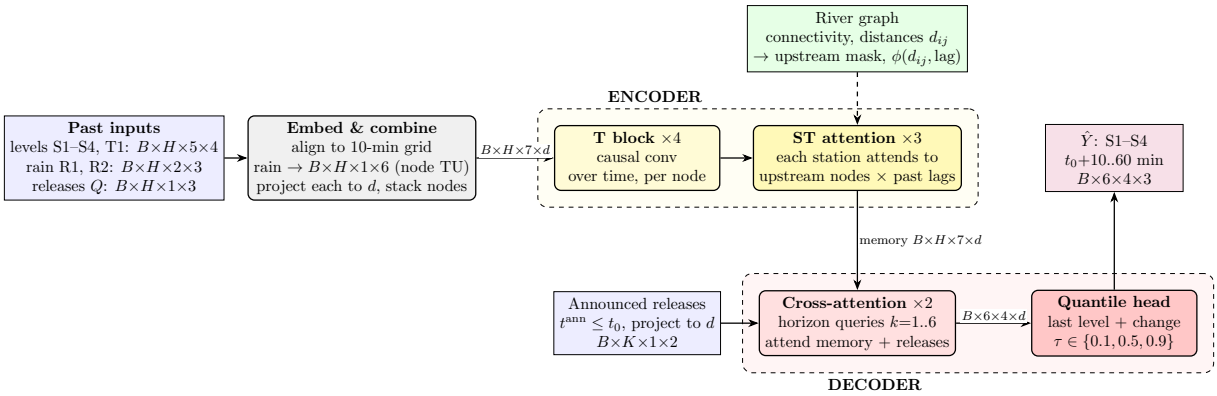


Figure 1: Encoder–decoder architecture. The encoder summarises the last 12 h of every node: a T block (causal convolution over time) followed by ST attention, where each station attends to its upstream nodes at all past lags, guided by the river graph. The decoder forms one query per horizon step and station, attends to the encoder memory and to announced releases, and outputs level quantiles as a change from the last observed level.

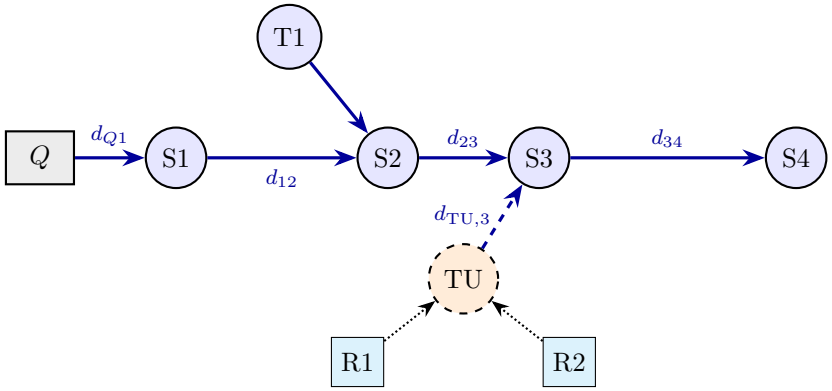


Figure 2: River graph. Solid: river edges with distances. Dotted: rain gauges feed only the virtual node TU (A7). $\mathcal{U}(n) = n$ plus every node with a path into n.

2. Assumptions

Supplied facts: 10-min levels at S1–S4 and T1; timestamped rain at R1, R2; recorded releases and schedules with announcement and effective times; topology and river distances. Not supplied: travel times, unmonitored inflow. Unavailable at issuance: future rain and levels.

A1 ≥3 years of data with several floods, so high-flow behaviour can be learned.

A2 Levels arrive within ∼20 min, rain within ∼30 min, and arrival times are logged, so inputs can be built from what had arrived.

A3 Gaps are short and faulty readings are detectable by range and rate checks, so masking is enough.

A4 Each rain report gives the depth since the previous one, so rain is summed per bin, not interpolated.

A5 Everything relevant to a 1-h forecast happened within the last 12 h, which sets H=72.

A6 Flow is downstream only (no backwater), so stations attend only upstream.

A7 R1 and R2 flank the unmonitored tributary (map), so rain enters only through TU and affects only S3 and S4. Other rain is buffered by the reservoir or already visible in T1.

A8 Unmonitored inflow is mainly rain-driven, so R1 and R2 are usable proxies.

A9 Travel time varies smoothly with flow and distance on a stable channel, so learned lags generalise.

A10 Release records are accurate and schedule versions are archived, needed for leak-free training.

A11 Each station has a fixed level datum, so the model can predict change from the last level.

A12 Future rain is unknown, so it is treated as uncertainty, not an input.

3. Forecasting task and data specification

$$\mathcal{I}(t_0) = \{r : t_r^{\text{obs}} \le t_0,\ t_r^{\text{arr}} \le t_0\} \cup \{\text{schedules with } t^{\text{ann}} \le t_0\},$$

$$\text{history } j = -71..0 \ (12\,\text{h}), \qquad \text{horizon } k = 1..6 \ \ (k{=}6:\ t_0{+}60 \text{ min}).$$

Forecasts are issued every 10 min at t_0 from $\mathcal{I}(t_0)$ only. Outputs are the quantiles $\tau \in \{0.1, 0.5, 0.9\}$ of the level at S1–S4.

Tensor	Dims	Axes / features	Coverage
X^h levels	$B{\times}H{\times}5{\times}4$	batch, time, node (S1–S4, T1); [level, δh, mask, age]	$(t_0{-}12\text{h}, t_0]$
X^r rain	$B{\times}H{\times}2{\times}3$	batch, time, gauge; [log(1+depth), mask, age]; reshaped to $B{\times}H{\times}1{\times}6$ as TU's features	$(t_0{-}12\text{h}, t_0]$
X^q past releases	$B{\times}H{\times}1{\times}3$	batch, time, Q; [actual, scheduled as-of, mask]	$(t_0{-}12\text{h}, t_0]$
X^s schedule	$B{\times}K{\times}1{\times}2$	batch, horizon, Q; [release, available]	$(t_0, t_0{+}1\text{h}]$, $t^{\text{ann}}{\le}t_0$
P; D, M	$N{\times}F_p$; $N{\times}N$	node static features; distances, upstream mask	static
Y, m	$B{\times}K{\times}4$	batch, horizon, station; target, observed-mask	$t_0{+}k\Delta$
$\hat{Y}$	$B{\times}K{\times}4{\times}3$	batch, horizon, station, quantile	issued at t_0

Alignment.

- *Time:* levels take the last value per 10-min bin; rain depth is spread over its report interval and summed per bin; releases are step-held. Masks and ages mark gaps.

- *Space:* no interpolation. Each source is a node (the two gauges are concatenated into TU), related only through $\mathcal{U}(n)$ and d_{ij}.

- *Features:* per-type projections W_c map every source to width d before stacking on the node

axis.

4. Model computations and design justification

Notation: $\mathcal{U}(n)$ = upstream closure (Figure 2), e.g. $\mathcal{U}(\text{S3}) = \{\text{S3}, \text{S2}, \text{S1}, \text{T1}, Q, \text{TU}\}$; $h_{t,n} \in \mathbb{R}^d$ is node n at time t. TU has no sensor, so $h_{t,\text{TU}}$ is a learned rain feature, not an inflow estimate; a tributary gauge would make it a supervised node without structural change.

Stage	**Formulation**
(a) Projection	$z^{(0)}_{t,n} = W_{c(n)} x_{t,n} + p_n + e_t$
(b) Temporal	$h^{(\ell)}_{t,n} = h^{(\ell-1)}_{t,n} + \text{MLP}\big(\sum_{j=0}^{2} W_j h^{(\ell-1)}_{t-j\delta_\ell,n}\big), \quad \delta_\ell = 1, 2, 4, 8$
(c) Lag attention	$\alpha_{t,j}(i,s) = \text{softmax}_{i\in\mathcal{U}(j),\, s\le t} \frac{(W_Q h_{t,j})^\top (W_K h_{s,i} + \phi(d_{ij}, t-s))}{\sqrt{d}}$ $h_{t,j} \mathrel{+}= \sum_{i,s} \alpha_{t,j}(i,s)\, W_V h_{s,i}$
(d) Decoder	$u_{k,n} \mathrel{+}= \text{Attn}\big(u_{k,n}; \{h_{s,i} : i \in \mathcal{U}(n)\} \cup \{f_{k'} : k' \le k\}\big), \quad f_{k'} = W_s x^s_{k'} + e_{k'}$
(e) Output	$\hat{y}^{(0.5)}_{k,n} = \bar{y}_n + \sigma_n a, \quad \hat{y}^{(0.9)} = \hat{y}^{(0.5)} + \sigma_n \,\text{softplus}(b), \quad \hat{y}^{(0.1)} = \hat{y}^{(0.5)} - \sigma_n \,\text{softplus}(c)$
(f) Loss	$\mathcal{L} = \frac{\sum w_b\, m\, \rho_\tau(y - \hat{y}^{(\tau)})}{\sum w_b\, m}, \quad \rho_\tau(r) = \max(\tau r, (\tau-1) r), \quad w_b = 1 + \lambda \mathbb{1}[\text{flood}]$

Table 1: Model components.

Why each block:

(b) Local trends and rain-accumulation shape.

(c) One layer reaches every upstream source at every lag (A5). Queries and keys depend on current states, so lags shift with flow (A9). ϕ handles uneven spacing; $\mathcal{U}$ encodes flow direction (A6).

(d) Announced releases act only on steps where they are already effective ($k' \le k$).

(e) Anchoring on $\bar{y}_n$ makes the model predict change from persistence (A11); softplus keeps quantiles from crossing.

(f) A point loss gives only a mean or median. Irreducible uncertainty (tributary, future rain) needs bands; w_b up-weights floods.

Training: chronological splits, as-of inputs, random masking, 5-seed ensemble, conformal calibration.

Source	Path to the forecast
Upstream stations	Keys in downstream lag attention (c), read at state-dependent lags
T1	In $\mathcal{U}$(S2, S3, S4); contribution learned
Rainfall	R1, R2 → TU (a, b); TU is a key for S3 and S4 only (c), at a learned catchment-plus-travel lag (A7, A8)
Releases	Past: via Q in (c). Announced future: decoder (d), mainly affecting S1
Imposed (physical)	Topology and direction; causality ($s \le t$, $k' \le k$, no future rain); distances; rain acts only via TU; rain as depth; level continuity
Learned	Travel times, attenuation, confluence mixing, rain → TU → S3 response, release response, band widths

5. Operation and exceptional conditions

Condition	Handling
(i) Missing	Value $\to 0$ with mask $= 1$ and growing age; $\bar{y}_n$ uses the last valid level. Training randomly drops points, blocks and whole stations. Missing targets are masked out of the loss. If all river data fail, fall back to persistence and alert.
(ii) Delayed	Only records with $t^{\text{arr}} \leq t_0$ are used; a late record counts as missing until it arrives and is used in the next 10-min cycle. Training replays history as-of each issuance with logged delays (A2).
(iii) Announced releases	Latest schedule with $t^{\text{ann}} \leq t_0$, step-held onto $k = 1..6$. Training uses archived versions, never actual future releases (that would be leakage).
(iv) Scheduled $\neq$ actual	The past input holds both actual and as-of scheduled releases, so the model learns how far to trust schedules. A live deviation appears through Q within one cycle. Optional $\pm 20\%$ release scenarios.

Feasibility. With $L = HN = 504$ tokens, even dense attention costs $2L^2 d \approx 3 \times 10^7$ MACs per layer, about 10^9 for the 5-member ensemble: under 1 s on a CPU, so ingestion dominates the 10-min cycle.

6. Uncertainty, limitations and additional information

Uncertainty is reported as calibrated 10–90% bands (quantiles, ensemble spread, conformal calibration).

Limitation	Impact	Response / unresolved
Unmonitored tributary	Unexplained rises at S3, S4	TU rain proxy, wider bands; storms missing both gauges unresolved
Changing travel times	Timing errors on rising limb	State-dependent lags; floods beyond record unresolved
No future rain	Fast rises under-forecast	Bands widen after rain; needs nowcasts
Rain elsewhere unmeasured (local runoff S2–S4, T1 and S1 catchments)	Sudden local rises seen only once they reach a station	Level histories, wider bands; largely unresolved
Few extremes, drift	Peaks under-estimated	Event weighting, monitoring, retraining
Schedule deviations	Errors at S1, S2	Deviation input, 10-min refresh
Sensor faults	Corrupted inputs and targets	QC and masks; slow drift partly unresolved

Request	Benefit	Usable without?
Gauge on unmonitored tributary	Supervises TU; removes the largest S3/S4 error	Yes (rain proxy)
Rain gauges on T1 catchment and main-river reach	Earlier warning of local runoff	Yes
Rain radar / nowcasts	Future-rain input	Yes, wider bands
Travel-time studies	Lag prior for ϕ; validation	Yes
Rating curves, flood thresholds	Discharge checks, warning levels	Yes
Telemetry delay and QC logs	Realistic delay simulation	Yes (A2, A3)
Archived schedule versions	Leak-free training	Partly (lower skill)
Reservoir operating rules	Anticipate unscheduled releases	Yes

Question 2 — Motion or Memory?

All transformation and window functions were checked before any experiment (Table 1); every row matches.

Table 1: Implementation checks on the synthetic tensors (indices from 0 in (d)–(e)).

Check	Expected	Ours
(a) $\max \lvert M(M(x)) - x \rvert$	0	0 (`torch.equal`: True)
(b) $\max \lvert R^4(x) - x \rvert$	0	0 (`torch.equal`: True)
(c) $\max \lvert MRM(x) - R^{-1}(x) \rvert$	0	0 (`torch.equal`: True)
(d) bright pixel of $M(p)$	(7, 10, 43)	(7, 10, 43)
(e) bright pixel of $R(p)$	(7, 43, 10)	(7, 43, 10)
(f) 1× input / target frames	1–5 / 6–10	1–5 / 6–10
(g) 2× input / target frames	1, 3, 5, 7, 9 / 11, 13, 15, 17, 19	1, 3, 5, 7, 9 / 11, 13, 15, 17, 19
(h) 3× input / target frames	1, 4, 7, 10, 13 / 16, 19	1, 4, 7, 10, 13 / 16, 19

1. Is the training data already symmetric?

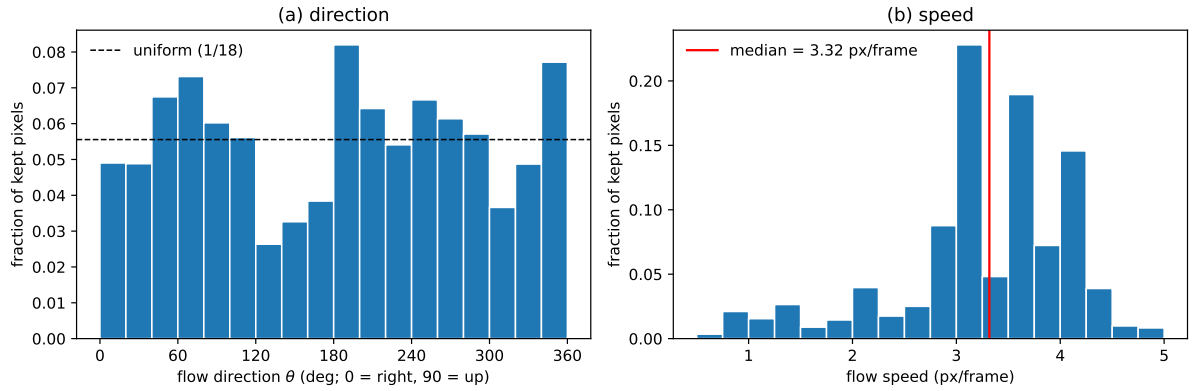


Figure 1: Farneback optical flow on the first 50 training videos (950 consecutive frame pairs), pooling 103,050 pixels with intensity > 0.2 in both frames and flow magnitude > 0.5 px. (a) Flow direction (0° = right, 90° = up; dashed line = uniform 1/18). (b) Flow speed; median 3.32 px/frame (red line). Both shown as fractions of kept pixels.

Direction. Every direction is well represented (Figure 1a). A mirror ($\theta \to 180° - \theta$) or rotation ($\theta \to \theta + 90°$) only reorders the bars, changing the histogram about as much as it already differs from uniform (table below). The bumps are mostly sampling noise from about 100 trajectories. **Mirrored and rotated trajectories are already familiar.**

Speed. Almost all motion lies between 3 and 4.5 px/frame (Figure 1b), so faster speeds are unseen:

Speed	Typical step (px/frame)	Range (px/frame)	Seen in training?
1×	3.3	3–4.5	yes
2×	6.6	6–9	no
3×	10.0	9–13.5	no

Overlap bias. Repeating the analysis on pairs with two separate bright blobs (a no-overlap heuristic) barely changes anything (table below). Overlap does not change the answer.

	All pairs	No-overlap pairs
Frame pairs	950	489
Pixels kept	103,050	51,532
Median speed (px/frame)	3.32	3.26
TV to uniform	0.109	0.106
TV original vs. mirrored	0.132	0.148
TV original vs. rotated 90°	0.200	0.208

TV = total-variation distance between direction histograms (0 = identical).

2. Motion, appearance, or both?

All three keep the motion physically valid; they differ in what they change:

Transformation	Motion statistics	Digit appearance
Mirror M	Unchanged: directions map $\theta \to 180° - \theta$, and these are already common in training.	Changed: a mirrored digit is not a real digit.
Rotation R	Unchanged: directions map $\theta \to \theta + 90°$, and these are already common in training.	Changed more: rotated digits look even less familiar.
Speed-up k×	Changed: steps are k times longer, outside the training range.	Unchanged: same digits.

Why this is a controlled experiment. Each transformation changes mainly one factor:

- Mirror and rotation change only appearance, so failures there point to appearance.
- Speed-up changes only motion, so failures there point to motion (unfamiliar speed or limited reach).
- Mirror and rotation give two levels of appearance change, so we can check whether a larger change causes a larger error.

The correct answer is always known exactly ($T(Y)$ or the faster true frames).

3. How far can information travel in model A?

One folded cell is 4 pixels. Only spatial convolutions larger than 1×1 move information; pixel (un)shuffle, the 1×1 head and the element-wise gate and cell updates do not. Each 3×3 convolution moves information by at most 1 cell:

Path to $\hat{x}_{\tau+1}$	3×3 convolutions	Reach
$u_\tau \to$ layer 1 $\to$ layer 2 $\to$ output	2	2 cells = 8 px
$h^{(1)}_{\tau-1} \to$ layer 1 $\to$ layer 2 $\to$ output	2	2 cells = 8 px
$h^{(2)}_{\tau-1} \to$ layer 2 $\to$ output	1	1 cell = 4 px

Model A's reach is 8 px per step (2 convolutions $\times$ 1 cell $\times$ 4 px).

- The reach is per axis (a 3×3 kernel covers a square), so up to $8\sqrt{2} \approx 11$ px along a diagonal.

- Recurrence carries information further over several steps, but only 8 px per step. A digit moving further than that leaves the region its old position (in $h^{(1)}_{\tau-1}$) can reach in one update.

4. How good is model A?

Table 2: Test error at $1\times$, $t = 1$. Model A: mean (seed 0, seed 1).

Model	MSE_1	MSE_2	MSE_3	MSE_4	MSE_5	$E^{(5)}$	Params
A	0.0172 (0.0175, 0.0170)	0.0205 (0.0208, 0.0203)	0.0228 (0.0231, 0.0225)	0.0245 (0.0248, 0.0242)	0.0262 (0.0264, 0.0259)	0.0222 (0.0225, 0.0220)	480,784
Blank	0.0426	0.0426	0.0426	0.0425	0.0425	0.0426	0
Copy-last	0.0492	0.0604	0.0669	0.0708	0.0732	0.0641	0

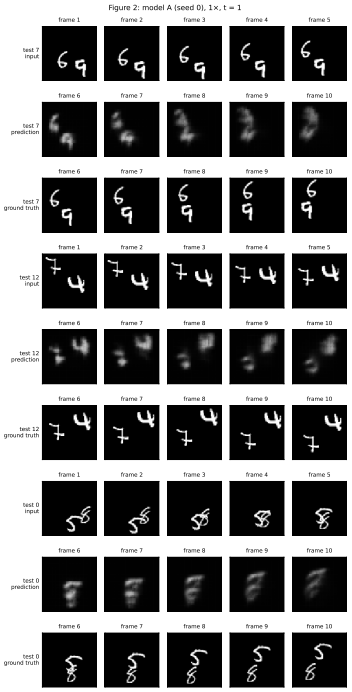


Figure 2: Input (frames 1–5), prediction and ground truth (frames 6–10) of seed-0 model A for test videos 7, 12 and 0 (dataset indices 9007, 9012 and 9000). Video 0 has overlapping digits.

Against the references. Model A roughly halves Blank's error and is far better than Copy-last; the seeds agree closely (Table 2). Copy-last is worse than Blank because digits move further than a stroke is wide: a copied frame misses the digit's new position *and* draws its old one.

Error growth. The error grows with the horizon, but more slowly at each step:

	$h = 1 \to 2$	$h = 2 \to 3$	$h = 3 \to 4$	$h = 4 \to 5$
Increase in MSE_h	+0.0033	+0.0023	+0.0017	+0.0017

Figure 2 shows why: positions stay roughly right (in video 12 the 4 even turns at the top wall), but shapes blur into smudges as the model feeds back its own predictions. Under MSE, dim brightness over likely positions costs less than a sharp digit in the wrong place, so the error levels off instead of exploding.

Overlapping digits. In video 0 the 5 and 8 merge and then separate. The model predicts one merged blob that drifts up and fades: the 5 survives as a smear, the 8 is lost. With no notion of separate objects, it cannot tell which strokes belong to which digit.

5. Predictions before testing

This answer was written and committed before any experiment in Testing the Baseline, and is reproduced exactly as first written.

Mirror and rotation. I predict model A will mostly pass the mirror test and do somewhat worse on the rotation test. From Figure 1(a), all 18 directions have motion and no direction is missing or rare, so every direction that a mirrored or rotated video produces is already well represented in training. The motion itself should be familiar, and digits should still move in the correct direction. We expect any errors to come from appearance: a mirrored/rotated digit is not a real digit, but rotated digits should seem less familiar than mirrored ones. I therefore expect C_M and E_M only slightly above $E^{(5)}$ at 1×, and C_R and E_R larger, with errors showing up as blurred or distorted digit shapes rather than wrong motion.

Speed. My reach estimate is 8 pixels per step per axis (2 convolutions × 1 cell × 4 pixels per cell). From Figure 1(b), motion is concentrated between 3 and 4.5 pixels per frame (median 3.32). So 1× (3–4.5 px) is well within reach. 2× (6–9 px) straddles the 8-pixel limit and is unseen in training, so I expect noticeable degradation but not complete failure. 3× (9–13.5 px) is beyond reach and unseen in training, so I predict model A breaks down at 3×: digits will blur, stall or vanish instead of moving to the right place. Because the reach is per axis, diagonal motion at 3× may degrade somewhat less than motion along the axes.

6. Why is consistency alone not enough?

Table 3: Consistency gap C_T and transformed error E_T at 1×, $t = 1$, on the test set. Model A: mean (seed 0, seed 1).

Model	$E^{(5)}$	C_M	E_M	C_R	E_R
A	0.0222 (0.0225, 0.0220)	0.00163 (0.00155, 0.00171)	0.0228 (0.0231, 0.0226)	0.00288 (0.00254, 0.00321)	0.0240 (0.0240, 0.0239)
Blank	0.0426	0	0.0426	0	0.0426
Copy-last	0.0641	0	0.0641	0	0.0641

Why the references are exactly consistent. Blank outputs zeros, and a transformed zero frame is still zero. Copy-last repeats the last frame, and T acts on each frame separately,

so transforming before or after copying gives the same result. Neither uses any knowledge of motion.

Reading C_T alone. C_T only checks whether the model agrees with itself, not whether it is right. A model can be consistent because it learned a symmetric motion rule, or because it ignores the content.

Why E_T is needed. E_T measures correctness on the transformed clip. In Table 3 the references have perfect consistency but the worst errors, while model A is slightly inconsistent but far more accurate. C_T alone would rank the useless references first; only the pair (C_T, E_T) shows a model that is both consistent and correct.

7. How does the motion change under a transformation?

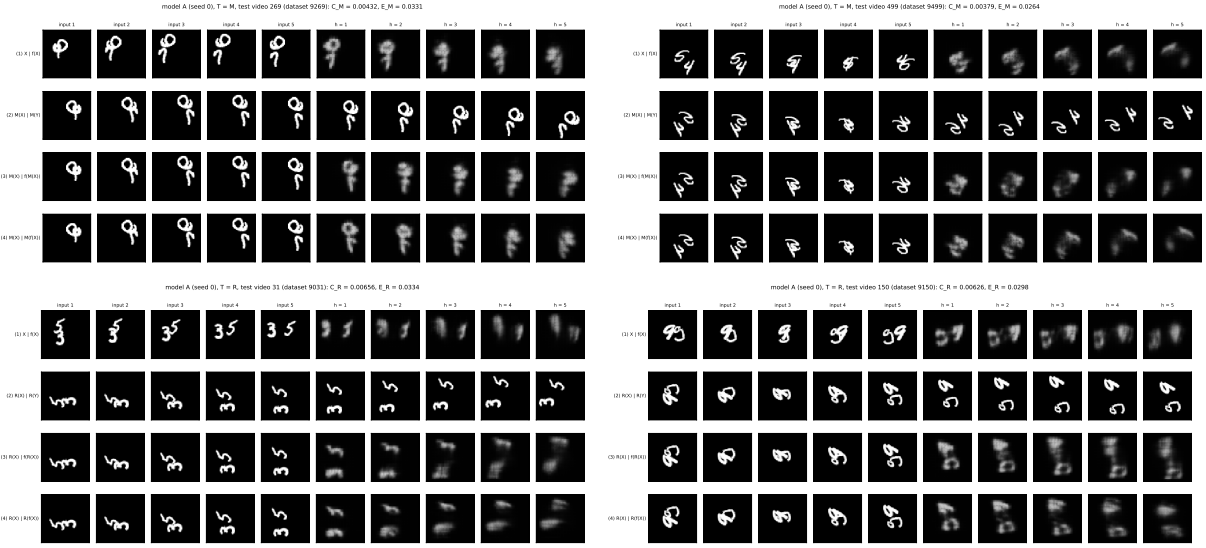


Figure 3: The two test videos with the largest per-video C_M and the two with the largest C_R for seed-0 model A (panels a–d from left to right and top to bottom). Rows: (1) $X \mid f(X)$; (2) $T(X) \mid T(Y)$; (3) $T(X) \mid f(T(X))$; (4) $T(X) \mid T(f(X))$. (a) $T = M$, test video 269, $C_M = 0.00432$, $E_M = 0.0331$; GIF: `gifs/fig3a_M_test269.gif`. (b) $T = M$, test video 499, $C_M = 0.00379$, $E_M = 0.0264$; GIF: `gifs/fig3b_M_test499.gif`. (c) $T = R$, test video 31, $C_R = 0.00656$, $E_R = 0.0334$; GIF: `gifs/fig3c_R_test31.gif`. (d) $T = R$, test video 150, $C_R = 0.00626$, $E_R = 0.0298$; GIF: `gifs/fig3d_R_test150.gif`.

We use test video 31 under rotation (Figure 3c), the video with the largest C_R.

Ground truth (row 2). Both rotated digits start near the bottom. The 5 moves up; the 3 moves down, bounces off the bottom wall and rises.

Same motion (rows 3 and 4). Both rows move both digits correctly, including the bounce.

Different shapes. Row 4 (original video, prediction rotated) draws smooth, elongated streaks. Row 3 (rotated video) draws rounder, blockier blobs with 4×4 patch texture, unrecognisable by $h = 5$. The same digits at the same positions are drawn differently depending only on orientation.

Conclusion. The motion rule carries over; the gap comes from appearance. The model has learned to draw upright digits and falls back to generic blobs for rotated ones, as expected from Q2.2.

8. Is rotation harder than mirroring?

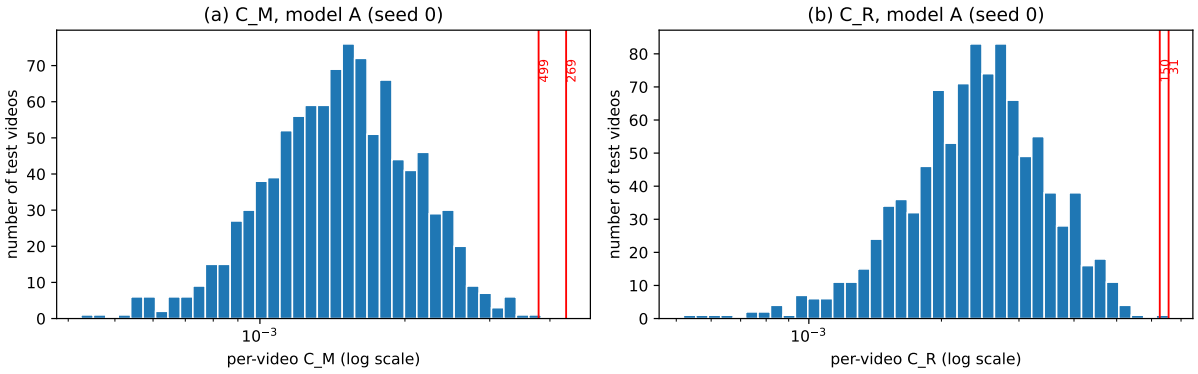


Figure 4: Histograms of per-video consistency gaps C_M (a) and C_R (b) of seed-0 model A over the 1,000 test videos (log scale). Red lines: the videos in Figure 3.

Yes. Rotation is worse than mirroring on every measure in Table 3, for both seeds:

	Mirror M	Rotation R	Ratio R/M
Consistency gap C_T	0.00163	0.00288	1.8
Extra error $E_T - E^{(5)}$	0.0006	0.0018	3.0

This is not driven by a few bad videos: the whole C_R distribution sits to the right of C_M (Figure 4), and the Figure 3 videos are just its tails.

Cause: appearance, not motion.

- *Motion is equally familiar* under both (Q2.1), and neither changes speed.
- *Appearance changes more* under rotation: a mirrored digit stays upright, a rotated one lies on its side (Q2.2).
- *Figure 3c agrees*: rotated digits move correctly but are drawn wrongly.

The larger appearance change causes the larger error, as predicted in Q2.5.

9. At what speed does model A fail?

Table 4: Model A at different speeds ($t = 1$, test set). Mean (seed 0, seed 1). Blank scores $E^{(2)} = 0.0426$ at every speed.

Model	$E^{(2)}$ 1×	$E^{(2)}$ 2×	$E^{(2)}$ 3×	$E^{(5)}$ 1×	$E^{(5)}$ 2×
A	0.0189 (0.0191, 0.0186)	0.0442 (0.0435, 0.0449)	0.0511 (0.0509, 0.0514)	0.0222 (0.0225, 0.0220)	0.0484 (0.0476, 0.0493)
Copy-last	0.0548	0.0661	0.0703	0.0641	0.0719

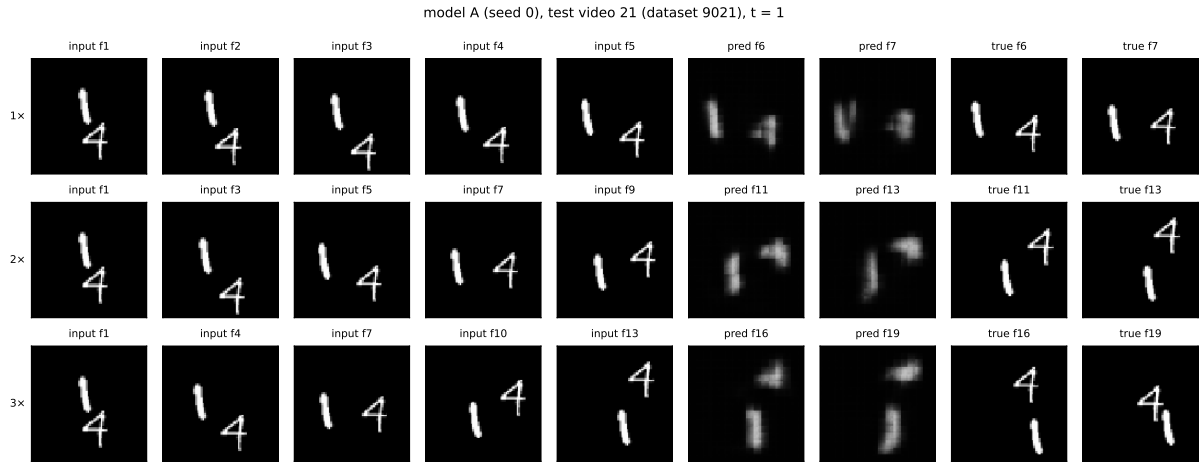


Figure 5: Model A (seed 0) on test video 21 (dataset index 9021) at 1×, 2× and 3× ($t = 1$): five input frames, then the first two predicted and true future frames. No digit overlap in any displayed frame (blob-count heuristic).

Model A already fails at 2×. Its error more than doubles from 1× to 2× and is worse than a blank frame at both 2× and 3× (Table 4). It only beats Copy-last because blur is penalised less than a sharp wrong digit.

Only partly consistent with our reach estimate. We expected the breakdown at 3×, but most 2× steps (about 6.6 px) are within 8 px. The estimate missed two things:

- *Reach is an upper bound.* A 6.6 px step relies on the weak outer taps of both kernels; the reach the model actually uses is likely smaller.

- *Reach is not the only limit.* Model A never saw steps above about 5 px (Figure 1b), so it never learned to produce them.

What the failure looks like (Figure 5).

- *Stalling:* at 2× and 3× the digits stay near their last input positions while the true digits move much further. The model applies roughly the short steps it learned at 1×, not the true velocity.

- *Blur:* shapes dissolve into soft blobs, the cheapest MSE hedge when the position is uncertain.

No digit is duplicated or vanishes: the model behaves like a blurred Copy-last, which is why its error lies between Blank and Copy-last.

10. Unfamiliar motion or limited reach?

From these results alone, we cannot tell. For model A, fast motion is both *unfamiliar* (never seen in training) and close to or beyond its *reach*. Both explanations predict the same errors and stalling (Table 4, Figure 5). The 2× failure hints at unfamiliarity, but reach is only an upper bound.

Proposed experiment. Change one factor at a time, keeping data, protocol and seeds fixed:

Model	Architecture	Training speeds	Tests
A	reach 8 px	1× only	baseline
C	reach 8 px (same as A)	1×, 2×, 3×	removes unfamiliarity only
D	reach $>$ 8 px, similar size	1×, 2×, 3×	also removes the reach limit

Compare $E^{(2)}$ at 2× (mostly within reach) and 3× (mostly beyond):

- *C fixes it:* unfamiliar motion.
- *C fails, D fixes it:* reach; more data cannot extend it.
- *Neither fixes it:* another limit (capacity, task difficulty).

D should match C's parameter count so that only reach changes.

11. Predictions before training B–D

This answer was written and committed before models B, C and D were trained, and is reproduced exactly as first written.

Model B. It is trained with mirror and rotation augmentation, and will reduce C_M, C_R, E_M and E_R, especially for rotation, because those gaps come from unfamiliar digit appearance, but it will not help at 2× or 3× since it still only sees 1× motion.

Model C. It is trained on 2× and 3× windows, will fix 2×, where motion is mostly within the 8-pixel reach and the failure is due to unfamiliar speed, but will still do poorly at 3× because those displacements exceed what the architecture can carry in one step.

Model D. It has a larger reach and the same speed augmentation, and will also fix 3x.

12. Model D: design and reach

Design. Model D is model A with **dilation 2** (padding 2) in the layer-2 ConvLSTM convolution; layer 1 keeps dilation 1. Everything else is unchanged: patch size 4, 64 channels per layer, one convolution for all gates, the 1×1 head, the 9-step rollout, and the training protocol (with speed augmentation, as for C). Dilation adds no weights, so D has **480,784** parameters (limit: 721,176). C and D therefore differ only in reach.

Reach. Same conventions as Q2.3; a dilation-2 convolution moves information by up to 2 cells:

Path to $\hat{x}_{\tau+1}$	Model A	Model D	Model D (px)
$u_\tau \to$ layer 1 $\to$ layer 2 $\to$ output	$1+1=2$ cells	$1+2=3$ cells	12
$h^{(1)}_{\tau-1} \to$ layer 1 $\to$ layer 2 $\to$ output	$1+1=2$ cells	$1+2=3$ cells	12
$h^{(2)}_{\tau-1} \to$ layer 2 $\to$ output	1 cell	2 cells	8
Longest path	**8 px**	**12 px**	

D's reach is 12 px per step, strictly larger than A's 8, and covers most of the 3× range.

Why mixed dilation. Offsets $\{0, \pm 1\}$ (layer 1) plus $\{0, \pm 2\}$ (layer 2) cover every offset from -3 to $+3$ cells. Dilation 2 in *both* layers would reach 16 px but only move information by even cell offsets, so a digit moving into the next cell (typical at 1×) could never be tracked ("gridding").

Selection (validation split only).

Candidate	Change from A	Params	Reach	$E^{(5)}$ 1× / $E^{(2)}$ 3× (seed 0; seed 1)	Outcome
C (reference)	none	480,784	8 px	0.0269 / 0.0330; 0.0260 / 0.0325	–
dil2	dilation 2 in both layers	480,784	16 px	stuck (10 epochs)	rejected: gridding
p8	patch size 8	594,496	16 px	stuck	rejected: did not train
k5h46	5×5 kernels, 46 channels	709,520	16 px	stuck	rejected: did not train
k5h40	5×5 kernels, 40 channels	544,976	16 px	0.0276 / 0.0309; stuck	rejected: unreliable
d12	**dilation 1, then 2**	**480,784**	**12 px**	**0.0274 / 0.0320; 0.0269 / 0.0320**	**chosen**

Validation split, $t = 1$, final epoch. Blank scores about 0.0395. "Stuck" = stayed at the Blank level for the whole run.

Several designs got stuck predicting near-blank frames, a known trap for MSE with a sigmoid output on sparse images. D must keep A's protocol, so we could not change the optimiser or initialisation; training reliably became part of the choice. d12 trained on both seeds, improved 3× over C on both, and changes only the reach.

13. Which failure has which cause?

Table 5: Models A–D on the test set ($t = 1$). Mean (seed 0, seed 1). C_T, E_T and $E^{(5)}$ at 1×; cross-speed comparisons use $E^{(2)}$. Hardware: Tesla T4 (training and inference).

Model	$E^{(5)}$ 1×	C_M	E_M	C_R	E_R	$E^{(2)}$ 1×	$E^{(2)}$ 2×	$E^{(2)}$ 3×	Params	Train s/epoch	Inference s
A	0.0222 (0.0225, 0.0220)	0.00163 (0.00155, 0.00171)	0.0228 (0.0231, 0.0226)	0.00288 (0.00254, 0.00321)	0.0240 (0.0240, 0.0239)	0.0189 (0.0191, 0.0186)	0.0442 (0.0435, 0.0449)	0.0511 (0.0509, 0.0514)	480,784	10.7 (10.7, 10.7)	0.401 (0.400, 0.402)
B	0.0229 (0.0233, 0.0224)	0.00111 (0.00106, 0.00115)	0.0229 (0.0234, 0.0224)	0.00134 (0.00141, 0.00128)	0.0228 (0.0232, 0.0223)	0.0192 (0.0197, 0.0187)	0.0431 (0.0433, 0.0429)	0.0490 (0.0496, 0.0483)	480,784	10.8 (10.7, 10.8)	0.425 (0.421, 0.429)
C	0.0267 (0.0272, 0.0263)	0.00135 (0.00117, 0.00154)	0.0269 (0.0273, 0.0265)	0.00258 (0.00220, 0.00295)	0.0279 (0.0282, 0.0276)	0.0232 (0.0237, 0.0227)	0.0288 (0.0292, 0.0285)	0.0333 (0.0335, 0.0331)	480,784	10.1 (10.1, 10.1)	0.422 (0.423, 0.422)
D	0.0275 (0.0277, 0.0272)	7.31×10^{-4} (7.97×10^{-4}, 6.66×10^{-4})	0.0276 (0.0279, 0.0274)	0.00176 (0.00169, 0.00182)	0.0284 (0.0285, 0.0283)	0.0248 (0.0249, 0.0246)	0.0290 (0.0292, 0.0288)	0.0326 (0.0326, 0.0325)	480,784	14.8 (14.7, 14.9)	0.602 (0.602, 0.602)

Each comparison changes one thing: A→B adds appearances, A→C adds faster motion, C→D adds reach. The labels follow from Table 5:

Failure	Label	Comparison	Measure	Before → After	Change
Mirror	appearance	A → B	C_M	0.00163 → 0.00111	−32%
		A → B	$E_M - E^{(5)}$	0.0006 → 0.0000	gap closed
Rotation	appearance	A → B	C_R	0.00288 → 0.00134	−53%
		A → B	$E_R - E^{(5)}$	0.0018 → 0.0000	gap closed
2×	unfamiliar motion	A → C	$E^{(2)}$ 2×	0.0442 → 0.0288	−35%
		C → D	$E^{(2)}$ 2×	0.0288 → 0.0290	none
3×	mostly unfamiliar motion	A → C	$E^{(2)}$ 3×	0.0511 → 0.0333	−35%
	(small part reach)	C → D	$E^{(2)}$ 3×	0.0333 → 0.0326	−2%

Mirror and rotation: appearance. B sees the same motion as A but with transformed digits, and that alone closes the accuracy gap and cuts the consistency gaps, most for rotation. Speed augmentation (C) does not.

2×: unfamiliar motion. C fixes it; extra reach (D) adds nothing. A's architecture could already carry this motion; it had just never seen it.

3×: mostly unfamiliar motion, a little reach. C removes most of the failure. D's further gain is small but consistent across both seeds. This contradicts our Q2.11 prediction that C would still fail at 3×.

If C had failed at 3× while D succeeded, the cause would be reach. More data cannot fix that: training only changes weights on paths that exist, and if no path links a digit's old

position to its new one within a step, no weights can. Only an architectural change adds such paths.

Why C coped anyway. Some 3× steps are within reach (slower and diagonal motion); over two steps the state can spread a digit's position into a blurry region; and a blurry guess already scores well under MSE. C and D are still clearly worse at 3× than at 1×; since D barely helps, this remaining gap is likely task difficulty (longer steps, more bounces) and scarce 3× training windows, not reach.

14. Did robustness cost accuracy at 1×?

Yes; the size depends on the change:

Model	$E^{(5)}$ 1×	Change	Average $E^{(2)}$	Change
A	0.0222	–	0.0381	–
B	0.0229	+3%	0.0371	−3%
C	0.0267	+20%	**0.0284**	−25%
D	0.0275	+24%	0.0288	−24%

From Table 5. Change relative to A. Average $E^{(2)}$ = mean over 1×, 2×, 3×.

Why. All four models have the same capacity, but B covers eight orientations and C and D cover three speeds; C and D also see 1× in only a third of their windows.

Was it worth it?

- *B: yes.* A small 1× cost buys closed accuracy gaps and much smaller consistency gaps.
- *C: yes, unless only 1× matters.* A is worse than a blank frame whenever the speed changes; C is robust across speeds.
- *D over C: hardly.* Its 3× gain is outweighed by its 1× loss, and dilated convolutions make it slower (Table 5). Its value here is as evidence that reach was not A's main limit.

15. Learned versus built-in symmetry

Table 6: Symmetrised models on the test set ($t = 1$). Mean (seed 0, seed 1). Training time is that of the underlying model; symmetrisation needs no extra training. Inference includes all eight transformations, forward passes, inverse transformations and the averaging. Final model: symmetrised D.

Model	$E^{(5)}$ 1×	C_M	E_M	C_R	E_R	$E^{(2)}$ 1×	$E^{(2)}$ 2×	$E^{(2)}$ 3×	Params	Train s/epoch	Inference s
sym B	0.0223 (0.0227, 0.0218)	7.63×10^{-17} (8.24×10^{-17}, 7.03×10^{-17})	0.0223 (0.0227, 0.0218)	6.17×10^{-17} (6.68×10^{-17}, 5.67×10^{-17})	0.0223 (0.0227, 0.0218)	0.0187 (0.0192, 0.0183)	0.0425 (0.0427, 0.0423)	0.0482 (0.0488, 0.0476)	480,784	10.8 (10.7, 10.8)	3.42 (3.45, 3.40)
sym D	0.0274 (0.0276, 0.0272)	5.93×10^{-17} (5.48×10^{-17}, 6.38×10^{-17})	0.0274 (0.0276, 0.0272)	4.62×10^{-17} (4.28×10^{-17}, 4.96×10^{-17})	0.0274 (0.0276, 0.0272)	0.0247 (0.0248, 0.0246)	0.0290 (0.0292, 0.0289)	0.0325 (0.0325, 0.0325)	480,784	14.8 (14.7, 14.9)	4.84 (4.83, 4.84)
sym D+B	0.0273 (0.0272, 0.0274)	7.30×10^{-17} (6.30×10^{-17}, 8.30×10^{-17})	0.0273 (0.0272, 0.0274)	5.82×10^{-17} (5.05×10^{-17}, 6.60×10^{-17})	0.0273 (0.0272, 0.0274)	0.0242 (0.0243, 0.0242)	0.0290 (0.0292, 0.0287)	0.0325 (0.0325, 0.0324)	480,784	15.9 (15.6, 16.1)	4.80 (4.72, 4.88)

Why built-in symmetry is exact. For $\bar{f}(X) = \frac{1}{8}\sum_{g\in G} g^{-1}\big(f(g(X))\big)$ and $T \in G$, substitute $g' = gT$ (so $g^{-1} = Tg'^{-1}$):

$$\bar{f}(T(X)) = \frac{1}{8}\sum_{g\in G} g^{-1}\big(f(gT(X))\big) = \frac{1}{8}\sum_{g'\in G} T\,g'^{-1}\big(f(g'(X))\big) = T\big(\bar{f}(X)\big).$$

g' runs over all of G because G is a group, and T (a pixel permutation) moves out of the average. So $C_M = C_R = 0$ for *any* f; the measured $\sim 10^{-16}$ is rounding. Since T only permutes pixels, also $E_T = E^{(5)}$ exactly (Table 6).

Comparison.

Model	Symmetry	C_M	C_R	$E^{(5)}$	E_R	Inference s
A	none	0.00163	0.00288	0.0222	0.0240	0.401
B	learned	0.00111	0.00134	0.0229	0.0228	0.425
sym B	built in	$\approx 10^{-16}$	$\approx 10^{-16}$	0.0223	0.0223	3.42
D	none	7.31×10^{-4}	0.00176	0.0275	0.0284	0.602
D+B	learned	8.47×10^{-4}	7.85×10^{-4}	0.0276	0.0276	0.576
sym D	built in	$\approx 10^{-16}$	$\approx 10^{-16}$	0.0274	0.0274	4.84

Means over seeds, test set, 1×; from Tables 5 and 6 and the D+B evaluation.

- *Consistency:* learned symmetry only shrinks the gaps (D+B's C_M even rises); augmentation gives no guarantee. Built-in symmetry is exact.

- *Original clips:* built-in symmetry costs nothing and slightly helps; learned symmetry costs a little for B.

- *Transformed clips:* built-in gives $E_T = E^{(5)}$ by construction; learned gets close.

- *Inference:* built-in needs eight forward passes ($\approx 8\times$ slower); learned costs nothing extra.

When averaging hurts. If the base model is confident but inconsistent (sharp digits in different places per orientation), the average shows faint ghost copies. It would also force a wrong symmetry if the true dynamics were not symmetric (e.g. gravity). **Here it did not hurt**: every symmetrised model matches or beats its base model. The base models' C_T is small next to their error, so the eight predictions mostly agree and averaging acts like a small ensemble.

16. Does the final model fix the Q2.7 video?

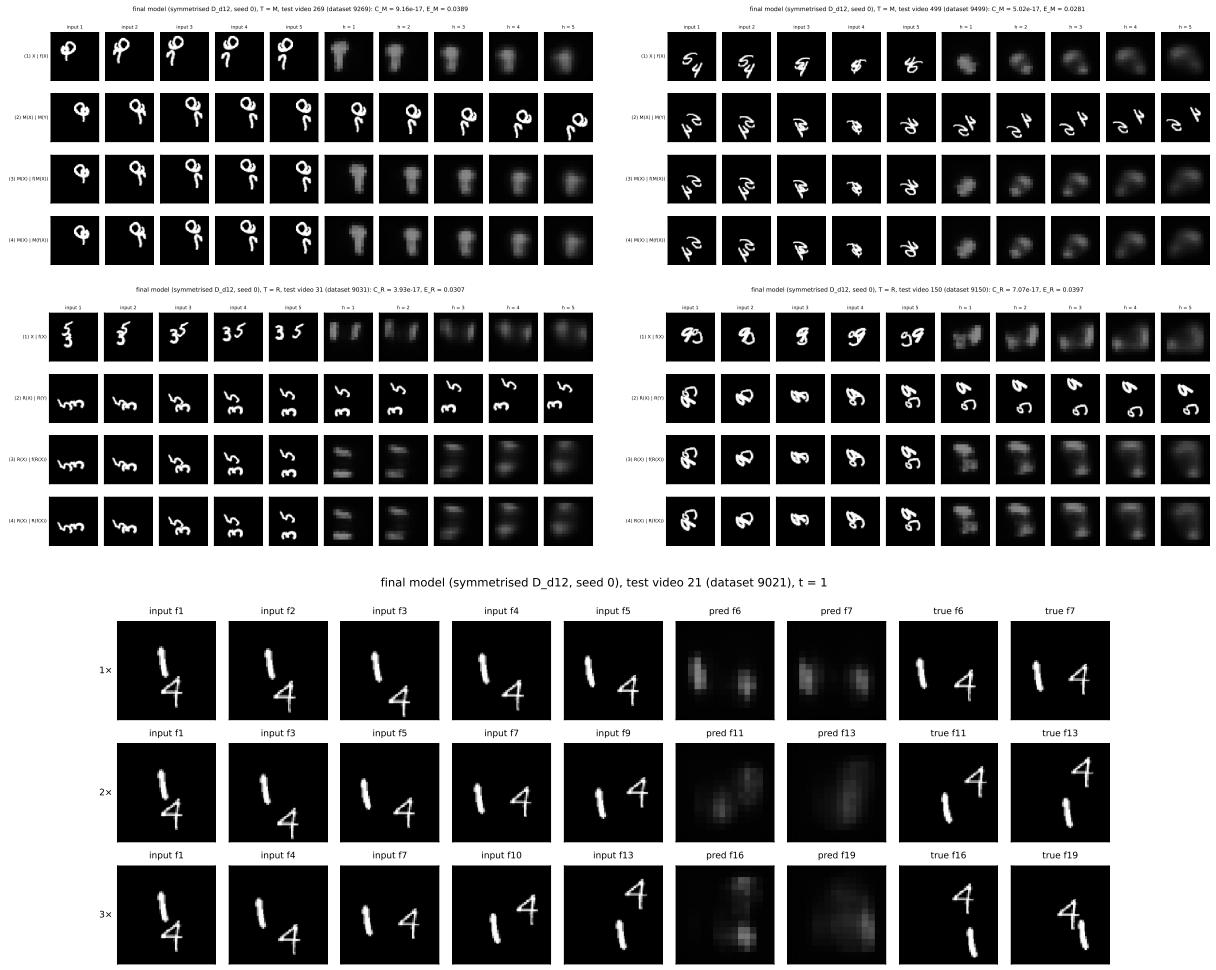


Figure 6: Final model (symmetrised D, seed 0). (a) Figure 3 repeated on the same videos (top two rows, panels a–d from left to right and top to bottom); rows as in Figure 3. Panel a: $T = M$, test video 269, $C_M = 9.16 \times 10^{-17}$, $E_M = 0.0389$; GIF: `gifs/fig6a_a_M_test269.gif`. Panel b: $T = M$, test video 499, $C_M = 5.02 \times 10^{-17}$, $E_M = 0.0281$; GIF: `gifs/fig6a_b_M_test499.gif`. Panel c: $T = R$, test video 31, $C_R = 3.93 \times 10^{-17}$, $E_R = 0.0307$; GIF: `gifs/fig6a_c_R_test31.gif`. Panel d: $T = R$, test video 150, $C_R = 7.07 \times 10^{-17}$, $E_R = 0.0397$; GIF: `gifs/fig6a_d_R_test150.gif`. (b) Bottom: Figure 5 repeated on test video 21 (dataset index 9021) at 1×, 2× and 3× ($t = 1$).

Test video 31 under rotation (Figure 6a, panel c; compare Figure 3c):

Video 31, $T = R$	Model A	Final model	Change
Per-video C_R	0.00656	3.93×10^{-17}	exact
Per-video E_R	0.0334	0.0307	-8%

- *Consistency: fixed exactly*, by symmetrisation. Rows 3 and 4 are now identical: the answer cannot depend on orientation.

- *Motion: still correct.* The 5 rises and the 3 bounces, as with model A.

- *Shape: not fixed.* The digits are softer, blockier patches than with model A, because D is

less sharp at 1× (Table 5) and averaging removes detail the orientations disagree on. E_R still improves, since a soft patch in the right place costs less under MSE.

- *Speed (Figure 6b):* at 3× the blobs no longer stall at the last input positions (compare Figure 5), thanks to speed augmentation and, slightly, D's reach. They are very blurry and spread out by frame 19.

Summary. Symmetrisation fixed the inconsistency, speed training fixed the stalling, and nothing fixed the blur: the final model knows where the digits go, but not what they look like.

Generative AI Usage

Generative AI tools were utilized during the completion of this assignment strictly as supplementary assistants. Primarily, AI was used as a learning aid to deepen my understanding of the assignment's underlying concepts, as well as to break down the specific architectures and techniques required for the tasks.

Furthermore, these tools provided support in the drafting and grammatical refinement of this report, debugging and structuring programming code, and compiling supporting evidence. However, all core ideas, critical analyses, experimental methodologies, and final implementations presented in this document are entirely my own original work. The AI was leveraged to facilitate execution and conceptual clarity, not to replace independent critical thinking or problem-solving.